

7.3. Escritura de información binaria en ficheros.....	1
7.4. Escritura de información en ficheros de texto.....	2
7.4.1. FileOutputStream, OutputStreamWriter y BufferedWriter.....	2
7.4.2. FileWriter y PrintWriter.....	3

7.3. Escritura de información binaria en ficheros

En el caso de la escritura binaria de ficheros haremos uso de la clase `java.io.FileOutputStream`, clase hija de `java.io.OutputStream`.

A la hora de crear el objeto de la clase `FileOutputStream` contamos con varias opciones.

Por un lado, podemos elegir, como ocurría con `FileInputStream`, entre pasar como parámetro un objeto `File` o directamente la ruta del fichero que queremos utilizar.

```
public FileOutputStream(File file) throws FileNotFoundException
```

```
public FileOutputStream(String name) throws FileNotFoundException
```

Con estos dos constructores, en el caso de que el fichero ya existe, se eliminará todo su contenido previamente a realizar la escritura. Es decir, que perderemos todo el contenido que tenía antes de escribir sobre él.

En el caso de que lo que necesitemos sea añadir contenido, tenemos dos constructores, con los que podemos indicar esta opción.

```
public FileOutputStream(File file, boolean append) throws  
FileNotFoundException
```

```
public FileOutputStream(String name, boolean append) throws  
FileNotFoundException
```

Estos dos constructores incluyen un parámetro boolean, denominado `append`, de tal forma que, si le pasamos `true`, lo que escribamos se añadirá al final del fichero, pero si pasamos `false`, se eliminará el contenido actual y se escribirá sobre el fichero vacío.

Para escribir en el fichero, utilizaremos el método `write`, al cual podremos pasarle como parámetro un array de bytes o un entero correspondiente al valor de un byte.

```
byte[] array = new byte[26];

File archivo = new File("archivo02.txt");
FileOutputStream escritor1 = new FileOutputStream (archivo);
for(int i = 0; i < 26; i++){
    escritor1.write(i);
    array[i] = (byte) i;
}
escritor1.close();

FileOutputStream escritor2 = new FileOutputStream ("archivo02.txt",
true);
escritor2.write(array);
escritor2.close();
```

7.4. Escritura de información en ficheros de texto

Al igual que ocurría con la lectura de ficheros de texto, en la escritura tenemos dos opciones.

7.4.1. FileOutputStream, OutputStreamWriter y BufferedWriter

La primera es utilizando `java.io.FileOutputStream`, combinado con `java.io.OutputStreamWriter` y `java.io.BufferedWriter`.

El procedimiento consiste en crear un objeto `FileOutputStream`, utilizar este objeto para crear un objeto `OutputStreamWriter`, y después utilizar este último para crear un `BufferedWriter`.

```
public OutputStreamWriter(OutputStream out)
```

```
public BufferedWriter(Writer out)
```

El método `write()` será el encargado de la propia escritura en el fichero. A este es método podremos pasarle por parámetro tanto un carácter, como una cadena de caracteres.

```
FileOutputStream fos = new FileOutputStream("archivo01.txt");
OutputStreamWriter osw = new OutputStreamWriter(fos);
BufferedWriter bw = new BufferedWriter(osw);
```

```
String[] cadenas = {"Hola,", "¿qué", "tal?"};

for(String cadena:cadenas){
    fw.write(cadena + " ");
}
fw.write("\n");
for(String cadena:cadenas){
    fw.write(cadena + "\n");
}
bw.close();
osw.close();
fos.close();
```

7.4.2. FileWriter y PrintWriter

La segunda opción es utilizar la clase `java.io.PrintWriter` en combinación con `java.io.FileWriter`.

`FileWriter` cuenta con cuatro constructores que nos permiten pasar como parámetro un objeto `File`, un `String` con la ruta del fichero, y también nos da la opción de indicar si queremos eliminar el contenido del fichero, si ya existe, o si por el contrario deseamos añadir la nueva información al final del mismo.

```
public FileWriter(File file) throws IOException
```

```
public FileWriter(File file, boolean append) throws IOException
```

```
public FileWriter(String fileName) throws IOException
```

```
public FileWriter(String fileName, boolean append) throws IOException
```

Al constructor de `PrintWriter` le vamos a pasar el objeto `FileWriter` que obtengamos.

```
public PrintWriter(Writer out)
```

Como métodos de escritura, `PrintWriter` cuenta con una gran cantidad, entre los que destacan:

- **print**: que nos permitirá escribir en el fichero datos de tipo primitivo como caracteres, enteros, números en coma flotante, ... y también strings.
- **println**: que funciona igual que "print", pero este escribirá un salto de línea al final.
- **write**: con él podemos escribir caracteres, arrays de caracteres y Strings.

```
FileWriter archivo = new FileWriter("archivo0.txt", false);
PrintWriter escritor = new PrintWriter(archivo);
String[] cadenas = {"Hola,", "¿qué", "tal?"};

for(String cadena:cadenas){
    escritor.print(cadena + " ");
}
escritor.println("");
for(String cadena:cadenas){
    escritor.println(cadena);
}
escritor.close();
```

A continuación, tenéis los links a la ayuda de java de todas las clases que hemos visto para la escritura:

<https://docs.oracle.com/javase/8/docs/api/java/io/FileOutputStream.html>

<https://docs.oracle.com/javase/8/docs/api/java/io/OutputStreamWriter.html>

<https://docs.oracle.com/javase/8/docs/api/java/io/BufferedWriter.html>

<https://docs.oracle.com/javase/8/docs/api/java/io/FileWriter.html>

<https://docs.oracle.com/javase/8/docs/api/java/io/PrintWriter.html>

Sigamos practicando con la escritura de ficheros:

